Retrieval-Augmented Generation for Question Answering

A Sample Research-Style Document for Testing a PDF-Based RAG Pipeline

Document type: Educational sample paper **Version:** 1.0

Abstract

Retrieval-Augmented Generation (RAG) is a system design pattern that combines information retrieval with a language model. Instead of relying only on information stored in a model's parameters, a RAG application retrieves relevant passages from an external collection and provides those passages as context when generating an answer. This sample document describes a typical RAG workflow, including document ingestion, text splitting, embedding generation, vector search, context assembly, and answer generation. It also outlines common failure modes and practical evaluation methods. This document is an illustrative teaching example, not a peer-reviewed scientific publication and not a report of new experiments.

Keywords

Retrieval-Augmented Generation; information retrieval; embeddings; vector databases; question answering; evaluation

1. Introduction

Large language models can produce fluent responses, but they may provide unsupported statements, omit details from a particular document, or lack access to newly updated information. A document-grounded question-answering system can reduce these problems by retrieving evidence from a specified knowledge collection before generating a response.

RAG separates the knowledge source from the language model. The source may contain policy manuals, technical documentation, research papers, product guides, or other text-based material. When a user asks a question, the system searches the collection for relevant passages and includes the strongest matches in the model's prompt. The response can then be grounded in those passages.

2. Background and Core Concepts

Document ingestion. Ingestion extracts text and useful metadata from source files. For PDF files, the extraction step should preserve page numbers and source filenames when possible. Empty, encrypted, or image-only PDFs may require separate handling.

Chunking. Long documents are commonly divided into smaller text segments called chunks. Chunks that are too large may contain unrelated information; chunks that are too small may lose context. Overlapping neighboring chunks can preserve continuity across boundaries.

Embeddings. An embedding model maps text to a numerical vector. Texts with related meanings may be near one another in the embedding space, allowing semantic retrieval even when the query and document use different words.

Vector search. The system compares a query vector with stored document vectors using a similarity measure such as cosine similarity or an index-specific distance metric. Retrieved chunks are then ranked and selected for the answer context.

Generation. A language model receives the user's question and the retrieved passages. A well-designed prompt instructs the model to answer from the evidence, acknowledge missing information, and avoid inventing details not supported by the source.

3. Typical RAG Architecture

Stage	Purpose	Example output
1. Load	Read source files and extract text	Text plus filename and page
2. Split	Divide text into manageable chunks	Overlapping text segments
3. Embed	Convert each chunk into a vector	Numerical vector representations
4. Index	Store vectors and associated metadata	Searchable vector index
5. Retrieve	Find passages relevant to a query	Top-k candidate chunks
6. Generate	Answer using retrieved context	Grounded natural-language response

4. Example Query Workflow

Suppose a user asks, “Why does a RAG system split a document into chunks?” The retriever searches the indexed passages and may return the chunk explaining that chunking limits context size and helps retrieve specific sections. The generation step uses that evidence to produce a concise answer. If no relevant passage is found, the system should state that the document does not contain enough information rather than fabricate a citation.

Illustrative answer: A RAG system splits documents into chunks so it can index and retrieve smaller, more focused passages. This can improve relevance and fit the retrieved evidence into the model's context window. Chunk size and overlap should be evaluated for the target documents and questions.

5. Common Failure Modes

Missing or empty files: A PDF file can exist on disk while containing zero bytes. File existence alone does not prove that the document can be read. A robust ingestion pipeline checks file size and catches parser errors.

Scanned PDFs: Some PDFs contain page images rather than embedded text. A standard text extractor may return empty strings for those pages; optical character recognition (OCR) may be needed.

Poor retrieval: Relevant evidence may not be returned if chunks are poorly sized, embeddings are unsuitable, metadata filters are incorrect, or the query is phrased differently from the source.

Unsupported answers: A generator may make claims beyond the retrieved evidence. Prompts, source citations, answer verification, and abstention behavior can help reduce this risk, but they do not guarantee perfect factuality.

6. Evaluation

A RAG application should be evaluated as both a retrieval system and an answer-generation system. Retrieval metrics can include Recall@k, Precision@k, and Mean Reciprocal Rank. Answer evaluation can examine factual correctness, relevance, completeness, citation support, and whether the system appropriately declines to answer when evidence is missing. A useful test set contains realistic questions, expected supporting passages, and reference answers.

Evaluation should include difficult cases: questions requiring information from multiple pages, questions whose answers are absent from the document, duplicate or contradictory passages, tables, and scanned pages. Latency, memory use, indexing cost, and update behavior also matter in deployed systems.

7. Practical Recommendations

A reliable prototype should log ingestion errors, retain source and page metadata, use consistent chunking and embedding settings, inspect retrieved passages during debugging, and separate retrieval failures from generation failures. For a small local project, a vector index can be sufficient; larger systems may need persistent storage,

access controls, incremental indexing, monitoring, and repeatable evaluation.

8. Conclusion

RAG connects language generation to an external evidence collection. Its quality depends on the complete pipeline: readable source files, meaningful chunks, useful embeddings, relevant retrieval, and evidence-aware generation. Testing each stage independently makes failures easier to diagnose and provides a stronger basis for improving a document question-answering application.

References and Further Reading

[1] Lewis, P. et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” Advances in Neural Information Processing Systems (NeurIPS).

[2] Karpukhin, V. et al. (2020). “Dense Passage Retrieval for Open-Domain Question Answering.” Proceedings of EMNLP.

[3] This sample document is an educational demonstration created for testing PDF ingestion and RAG workflows. It is not peer-reviewed and does not claim original experimental results.